



Handout

Internationale Hochschule Duales Studium

Studiengang: Informatik

MedTracker

Lukas Penner

Betreuer/in: Bernd Kruggel

Inhalt

1. Projektüberblick	1
2. Zielsetzung	1
3. Kernfunktionen.....	2
4. Systemarchitektur	2
5. Datenhaltung	3
6. Ablauf: Medikament anlegen	3
7. Ablauf: Erinnerungssystem.....	4
7.1. Beteiligte Komponenten.....	5
7.2. Ablauf im Detail.....	5
8. Technologiestack	6
9. Technische Besonderheiten	7
10. Stärken	8
11. Grenzen und Erweiterungspotenziale	8
12. Fazit	8

1. Projektüberblick

MedTracker ist eine native Android-Anwendung zur strukturierten Unterstützung der täglichen Medikamenteneinnahme. Die App ermöglicht es Nutzerinnen und Nutzern, Medikamente inklusive Dosierung, Zusatzhinweisen und beliebig vieler Einnahmezeitpunkte zu verwalten.

Eine zentrale Funktion ist die zeitgenaue Erinnerung mittels Alarm- und Benachrichtigungssystem, kombiniert mit der Möglichkeit, Einnahmen direkt zu bestätigen.

Die Anwendung wurde mit modernen Android-Technologien entwickelt:

- Kotlin
- Jetpack Compose
- Material 3

Der Fokus liegt auf:

- hoher Usability
- robuster Offline-Funktionalität
- zuverlässiger Alarmverarbeitung auch nach Systemereignissen (z. B. Neustart)

2. Zielsetzung

Ziel des Projekts ist die Lösung typischer Probleme im Medikationsalltag:

- Vergessen von Einnahmen
- fehlende Transparenz bei Dosierungen
- komplexe Zeitpläne mit mehreren Einnahmen täglich
- Verlust von Erinnerungen nach Geräte-Neustart

MedTracker adressiert diese Herausforderungen durch:

- vollständig lokale Verarbeitung (kein Backend notwendig)
- persistente und robuste Erinnerungslogik
- einfache und intuitive Bedienoberfläche

3. Kernfunktionen

- Verwaltung von Medikamenten (Name, Dosierung, Hinweise)
- Unterstützung mehrerer Einnahmezeiten pro Tag
- Markierung von Medikamenten als „genommen“
- Speicherung des letzten Einnahmezeitpunkts
- Präzise Erinnerungen über den Android AlarmManager
- Vollbild-Alarmansicht (auch über Sperrbildschirm)
- Automatisches Wiederherstellen aller Alarmer nach:
 - Geräte-Neustart
 - Locked Boot
 - App-Update

4. Systemarchitektur

Komponentenübersicht

- **UI-Schicht:** Compose-basierte Oberfläche (MainActivity)
- **Datenmodell:** Medication, IntakeTime
- **Persistenz:** SharedPreferences (JSON-basiert)
- **Logik:** ReminderScheduler
- **Event Handling:** BroadcastReceiver
- **Systemintegration:** BootCompletedReceiver



5. Datenhaltung

Die Anwendung nutzt **SharedPreferences** zur lokalen Speicherung.

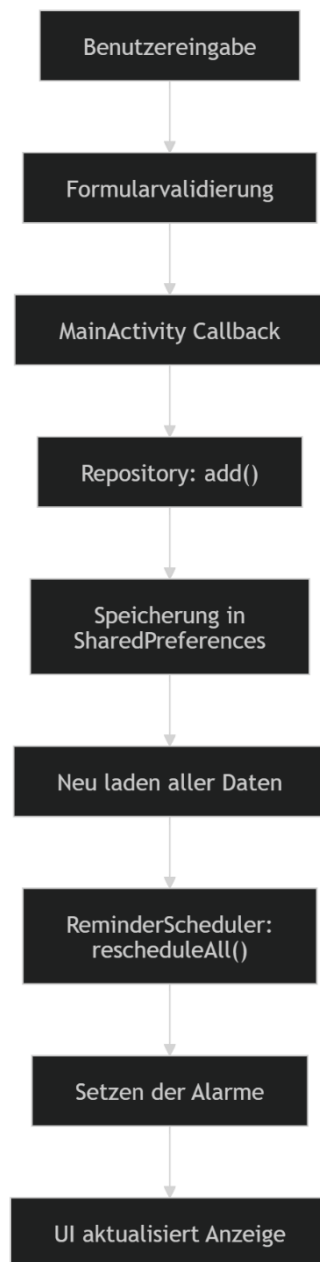
Datenstruktur eines Medikaments

```
{
  "id": String,
  "name": String,
  "dosage": String,
  "intakeTimes": [Time],
  "notes": String,
  "lastTakenAt": Timestamp
}
```

Bewertung

Vorteile	Nachteile / Perspektive
einfache Implementierung	eingeschränkte Abfragemöglichkeiten
keine externe Datenbank notwendig	keine Migrationen → Empfohlene Weiterentwicklung: Room Database
vollständige Offline-Fähigkeit	

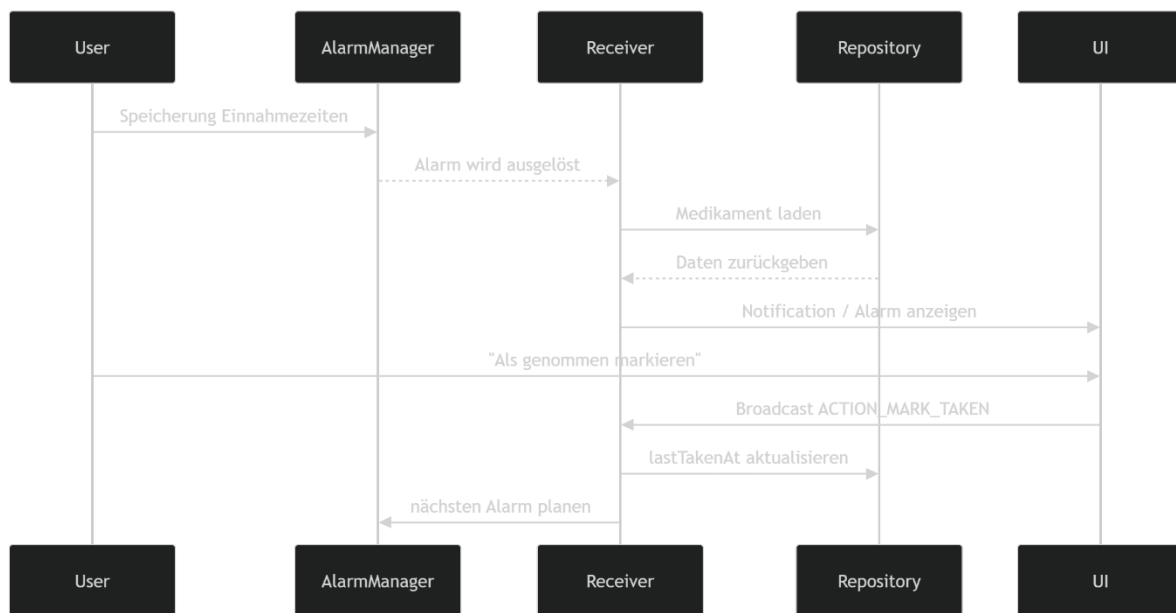
6. Ablauf: Medikament anlegen



Kernaspekt:

Nach jeder Änderung erfolgt ein vollständiges **Re-Scheduling aller Alarmer**, um Konsistenz sicherzustellen.

7. Ablauf: Erinnerungssystem



Das Sequenzdiagramm modelliert die Interaktion zwischen Benutzer, Systemkomponenten und Android-Systemdiensten beim Auslösen und Verarbeiten einer Medikamentenerinnerung.

7.1. Beteiligte Komponenten

- Benutzer (User)
- AlarmManager (Android-Systemdienst)
- MedicationAlarmReceiver (BroadcastReceiver)
- MedicationRepository (Datenzugriffsschicht)
- UI (Notification / AlarmAlertActivity)

7.2. Ablauf im Detail

1. Planung der Erinnerung

Zu Beginn speichert der Benutzer ein Medikament mit definierten Einnahmezeitpunkten. Diese Informationen werden an den AlarmManager übergeben, der für jeden Zeitpunkt einen separaten Alarm registriert.

2. Auslösen des Alarms

Sobald der geplante Zeitpunkt erreicht ist, sendet der AlarmManager ein Broadcast-Event. Dieses Event wird vom MedicationAlarmReceiver empfangen.

3. Laden der Medikamentendaten

Der Receiver benötigt Kontextinformationen zum Alarm (z. B. Medikament und Einnahmeslot). Daher erfolgt ein Zugriff auf das MedicationRepository, welches die entsprechenden Daten aus der lokalen Persistenz lädt.

4. Anzeige der Erinnerung

Nach erfolgreichem Laden der Daten stößt der Receiver die Benutzerinteraktion an:

- Anzeige einer Benachrichtigung
- optional Öffnen einer Vollbild-Alarmansicht (AlarmAlertActivity)

Ziel ist eine möglichst auffällige und unmittelbare Erinnerung.

5. Benutzerinteraktion

Der Benutzer reagiert auf die Erinnerung und wählt beispielsweise: „Als genommen markieren“

Diese Aktion wird als Broadcast (ACTION_MARK_TAKEN) zurück an den MedicationAlarmReceiver gesendet.

6. Aktualisierung des Zustands

Der Receiver verarbeitet die Aktion und aktualisiert im Repository den Wert:

- lastTakenAt (Zeitpunkt der Einnahme)

Damit wird der aktuelle Zustand des Medikaments persistiert.

7. Neuplanung des nächsten Alarms

Abschließend plant der Receiver direkt den nächsten Alarm für denselben Einnahmeslot:

- typischerweise für den nächsten Tag zur gleichen Uhrzeit

Dies stellt sicher, dass die Erinnerung zyklisch und zuverlässig weiterläuft.

Zentrale Eigenschaften des Ablaufs

- **Event-getrieben:** Reaktion erfolgt ausschließlich bei Alarm-Trigger
- **Entkopplung:** UI, Datenhaltung und Alarmsteuerung sind klar getrennt
- **Zustandskonsistenz:** Jede Nutzeraktion führt zu einer direkten Persistenzänderung
- **Selbstheilend:** Erinnerungen werden kontinuierlich neu geplant

8. Technologiestack

- **Sprache:** Kotlin
- **UI:** Jetpack Compose

- **Design:** Material 3
- **Build:** Gradle (Kotlin DSL)
- **Android APIs:**
 - AlarmManager
 - BroadcastReceiver
 - NotificationManager

Systemanforderungen:

- Minimum: Android 8 (API 26)
- Target: Android 15 (API 35)

9. Technische Besonderheiten

Berechtigungen

- POST_NOTIFICATIONS
- SCHEDULE_EXACT_ALARM
- RECEIVE_BOOT_COMPLETED
- USE_FULL_SCREEN_INTENT
- WAKE_LOCK

Zuverlässigkeitsmechanismen

Alarmer werden automatisch neu geplant bei:

- Geräte-Neustart
- Locked Boot
- App-Update (MY_PACKAGE_REPLACED)

Direct Boot Support

Verwendung von Device Protected Storage, um:

- Erinnerungen bereits vor Geräteentsperrung verfügbar zu machen

10. Stärken

- moderne, klare Benutzeroberfläche
- vollständige Offline-Funktionalität
- flexible Einnahmeplanung
- robuste Alarmarchitektur
- sauber strukturierter Code

11. Grenzen und Erweiterungspotenziale

Aktuelle Einschränkungen

- einfache Persistenzlösung (SharedPreferences)
- keine Historie der Einnahmen
- keine differenzierten Zeitpläne

Erweiterungsmöglichkeiten

- Migration zu Room Database
- Einführung von ViewModel + State Management
- Einnahmehistorie (Audit Trail)
- Wochenbasierte Einnahmepläne
- Snooze-/Reminder-Logik
- Backup- und Exportfunktionen
- Mehrbenutzer-/Pflegermodus

12. Fazit

MedTracker demonstriert eine praxisnahe Android-Anwendung, die zentrale Plattformmechanismen effektiv kombiniert:

- lokale Datenhaltung

- zuverlässige Alarmsteuerung
- reaktive Benutzeroberfläche

Die Architektur bietet eine solide Grundlage für Skalierung und Erweiterung und zeigt, wie sich mit nativen Android-Mitteln eine robuste, alltagstaugliche Gesundheitsanwendung realisieren lässt.